you can automate using BDD tools such as JBehave, Cucumber, and SpecFlow. When you automate these scenarios, they become executable specifications, but these will be reported as pending until you write the underlying code that actually exercises the application and verifies the outcomes. Once this happens, you can talk of automated acceptance tests.

Throughout the rest of this chapter, I'll refer to automated acceptance tests as *automated tests* or simply as *tests*. And for this chapter, *test suite* will refer to all of the automated acceptance tests for a project.

You're now ready to learn how to turn your pending executable specifications into full-fledged automated acceptance tests. But before we can discuss specific test automation approaches and technologies, we need to lay the groundwork and discuss how to ensure that these tests are implemented in a way that will be easy to extend and maintain as the test suite grows:

- A good acceptance test should communicate its intent clearly and provide meaningful feedback about the current state of the application. It should also be reliable and easy to maintain, so that the value provided by the test outweighs the cost of maintaining it.
- Techniques such as *initialization hooks* and *personas* can be used to ensure that the system is in a known state at the start of each test.
- Layers of abstraction can help make the tests clearer and easier to maintain by separating the *what* from the *how*.

Let's start off by discussing what makes an industrial-strength acceptance test.

7.1 Writing industrial-strength acceptance tests

It's not hard to write a simple automated test. There are plenty of books and resources on the internet that can show you how to write automated tests for all sorts of technologies, and we'll look at a few of these later on in this book. But what's harder is ensuring that your tests will stand the test of time. Will your tests continue to provide useful feedback several months down the track? Will they become harder to maintain as the code base grows? Will they become brittle and fragile, discouraging the team from fixing them when they break? Many teams spend a great deal of effort in building large and complex test suites, only to abandon them because they find that the cost of maintaining and updating them outweighs the value of the feedback they provide.

The tests in a well-written automated acceptance test suite should be either passing or pending. A failing automated acceptance test should be a red flag for the development team that demands immediate attention. But when tests fail too often due to sporadic technical issues, the team will lose confidence in them as a feedback mechanism and be less motivated to fix them when they break (see figure 7.3). This leads to a vicious circle, where there are always a few broken tests in the build. When this happens, the automated acceptance criteria no longer perform their primary role of providing feedback about the project's current health. This is what you want to avoid.